
Fantasizing with Dual GPs in Bayesian Optimization and Active Learning

Paul E. Chang*
Aalto University
Espoo, Finland
paul.chang@aalto.fi

Prakhar Verma
Aalto University
Espoo, Finland

ST John
Aalto University
Espoo, Finland

Victor Picheny
Secondmind.ai
Cambridge, UK

Henry Moss
Secondmind.ai
Cambridge, UK

Arno Solin
Aalto University
Espoo, Finland

Abstract

Gaussian processes (GPs) are the main surrogate functions used for sequential modelling such as Bayesian Optimization and Active Learning. Their drawbacks are poor scaling with data and the need to run an optimization loop when using a non-Gaussian likelihood. In this paper, we focus on ‘fantasizing’ batch acquisition functions that need the ability to condition on new fantasized data computationally efficiently. By using a sparse Dual GP parameterization, we gain linear scaling with batch size as well as one-step updates for non-Gaussian likelihoods, thus extending sparse models to greedy batch fantasizing acquisition functions.

1 Introduction

Bayesian Optimization (BO, [1]) is a popular technique for black-box optimization. Active Learning (AL, [2]) is the closely related problem of learning a decision boundary. AL and BO combine Gaussian process (GP) surrogate models to predict the quantity of interest with acquisition functions that encode trade-offs between exploration and exploitation. The maximization of these acquisition functions recommends a sequence of new points to query.

Standard GPs have a posterior distribution with a closed-form expression that can be efficiently updated [3] when new observations are added to the training data. This desirable updating property has been used to design BO and AL algorithms that query n_{new} points at once, by greedily building a batch of points, ‘fantasizing’ for each point an observation value and updating the posterior [4, 5]. Other BO and AL algorithms directly rely on update formulas to compute acquisition functions in closed form (e.g., [6, 7, 8]). Such formulas are not available in more complex scenarios, in particular sparse GPs [9] or for non-conjugate likelihoods. Sparse GP models are becoming increasingly popular as surrogate models for high-throughput BO loops [10, 11, 12] where data volumes are high.

Recently, Maddox et al. [13] proposed the OVC method for the fast conditioning of sparse variational GP models, building on streaming GP approximations [14]. They applied their OVC to evaluate existing batch acquisition functions [8, 15], showing superior performance to many existing batch BO methods. However, in order to apply their method in settings requiring non-Gaussian likelihoods, e.g. batch active learning strategies for binary classification, they must use an additional Laplace approximation with cost $O((m + n_{\text{new}})^3)$.

In this paper, we build on the work of [13] by looking at fast conditioning of sparse GPs, but instead using the dual parameter formulation of Adam et al. [16]. Dual-parameterized GPs were initially

*Work done while at Secondmind.ai

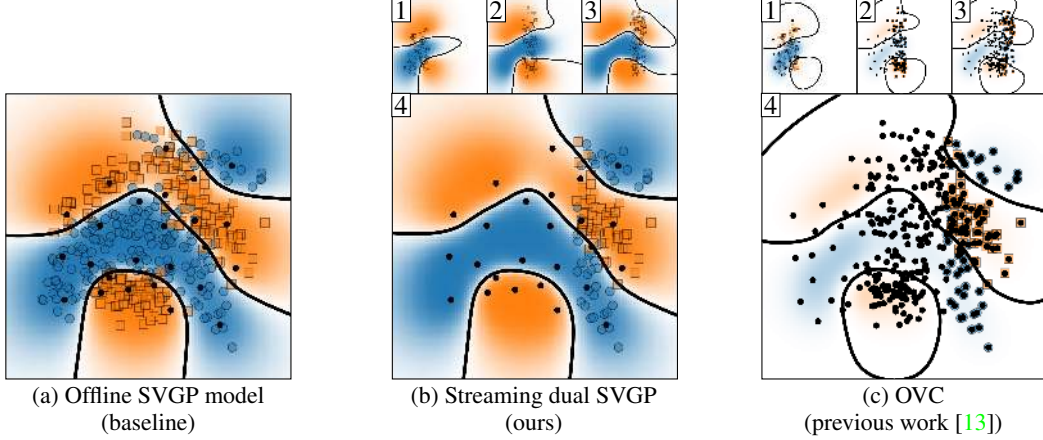


Figure 1: Dual conditioning on streaming banana data set; data $\blacksquare$ appears batch by batch (1–4). The plot shows the decision boundary — and the predictive class probability, with colour shading $\blacksquare$ and $\blacksquare$ increasing the more certain the model is about the class. The inducing points are overlaid as black dots. (a) Dual SVGP model trained with full data. (b) Dual SVGP model conditioned on the data appearing in batches. (c) Online Variational Conditioning (OVC, [13]) model on batched data.

motivated by a tighter hyperparameter bound and efficient natural gradient steps for optimizing the variational parameters. Here we show that they also offer decisive advantages for posterior update computations: no need for additional approximations, and the computational complexity $O(m^3 + m^2 n_{\text{new}})$ remains linear in the number of conditioning points. We find that we can perform effective one-step updates for most BO and AL problems, removing the need to run a variational optimization loop with non-Gaussian likelihoods.

2 Methods

Gaussian process models define a prior over (latent) functions $f \sim \mathcal{GP}(0, \kappa)$ that is characterized completely by a covariance function $\kappa(\mathbf{x}, \mathbf{x}')$. When a GP prior is combined with a data set $\mathcal{D} = (\mathbf{X}, \mathbf{y}) = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ of input–output pairs and a Gaussian likelihood, computational complexity of the posterior process is $O(n^3)$. If the likelihood is non-conjugate, a common technique is to perform variational inference. Fitting a Variational GP model (VGP, *e.g.* [17]) requires multiple $O(n^3)$ operations in an optimization loop until convergence. Sparse Variational GP (SVGP) models [9, 18] overcome the cubic scaling while simultaneously dealing with non-Gaussian likelihoods. They swap computations on the full training set $\mathbf{X}$ with a sparser set of *inducing points* $\mathbf{Z} := (\mathbf{z}_j)_{j=1}^m, (\mathbf{u})_j = f(\mathbf{z}_j)$, with the approximate posterior $q(\mathbf{u}; \mathbf{m}, \mathbf{V})$. The overall complexity is $O(nm^2)$, where $m \ll n$. The induced posterior of the function values at all other points is:

$$q_{\mathbf{u}}(\mathbf{f}) = \mathcal{N}(\mathbf{f}; \mathbf{A}\mathbf{m}^*, \mathbf{K}_{\mathbf{xx}} - \mathbf{A}\mathbf{K}_{\mathbf{zz}}^{-1}\mathbf{A}^\top + \mathbf{A}\mathbf{V}^*\mathbf{A}^\top), \quad (1)$$

where $\mathbf{K}_{\mathbf{xx}}$ is an $n \times n$ matrix with $\kappa(\mathbf{x}_i, \mathbf{x}_j)$ as the ij^{th} entry, $\mathbf{A} = \mathbf{K}_{\mathbf{xz}}\mathbf{K}_{\mathbf{zz}}^{-1}$. $\mathbf{K}_{\mathbf{xz}}$ and $\mathbf{K}_{\mathbf{zz}}$ are defined similarly to $\mathbf{K}_{\mathbf{xx}}$. To predict with an SVGP we need to infer the variational parameters $(\mathbf{m}^*, \mathbf{V}^*)$ at the optimum of the evidence lower bound [ELBO, 9]. The ELBO is also used to learn the hyperparameters θ of kernel and likelihood.

Adam et al. [16] showed that the optimal variational parameters $(\mathbf{m}^*, \mathbf{V}^*)$ are:

$$\mathbf{m}^* \equiv \mathbf{V}^* \boldsymbol{\lambda}^* \quad \text{and} \quad \mathbf{V}^* \equiv [\mathbf{K}_{\mathbf{zz}}^{-1} + \boldsymbol{\Lambda}^*]^{-1}. \quad (2)$$

The optimal dual variables $(\boldsymbol{\lambda}^*, \boldsymbol{\Lambda}^*)$ are found using the following iteration until convergence:

$$\begin{aligned} \boldsymbol{\lambda}_t &= (1 - \rho_t)\boldsymbol{\lambda}_{t-1} + \rho_t \nabla_{\boldsymbol{\mu}^{(1)}} \mathbb{E}_{q_{\mathbf{u}}(\mathbf{f})} [\log p(\mathbf{y} | \mathbf{f})], \\ \boldsymbol{\Lambda}_t &= (1 - \rho_t)\boldsymbol{\Lambda}_{t-1} + \rho_t \nabla_{\boldsymbol{\mu}^{(2)}} \mathbb{E}_{q_{\mathbf{u}}(\mathbf{f})} [\log p(\mathbf{y} | \mathbf{f})], \end{aligned} \quad (3)$$

where $(\boldsymbol{\mu}^{(1)}, \boldsymbol{\mu}^{(2)})$ are the expectation parameters of the approximating distribution $q(\mathbf{u})$, so $\boldsymbol{\mu}^{(1)} = \mathbf{m}$ and $\boldsymbol{\mu}^{(2)} = \mathbf{V} + \mathbf{m}\mathbf{m}^\top$, and $\boldsymbol{\lambda}_0$ and $\boldsymbol{\Lambda}_0$ are initialized to zero. As pointed out in [17], these are in

Algorithm 1 Fantasizing a batch with Dual Conditioning.

Input: $(\lambda^*, \Lambda^*), k, \alpha(\cdot)$ **Initialize:** $\mathbf{X}_b = \emptyset$

```
1: for  $i$  in  $1, 2, \dots, k$  do                                 $\triangleright k$  is desired number of query points
2:    $\mathbf{x}_i = \arg \max_{\mathbf{x}} \alpha(\mathbf{x})$                                  $\triangleright$  Calculate  $\alpha(\mathbf{x})$  using prediction function Eq. (1) at  $\mathbf{x}$ 
3:    $y_i = \mathbb{E}[f(\mathbf{x}_i)]$                                            $\triangleright$  Fantasized  $y$  is mean of the GP at  $\mathbf{x}_i$ 
4:    $\mathcal{D}_{\text{new}} = (\mathbf{x}_i, y_i)$                                  $\triangleright$  The fantasized data point is treated as new data
5:   Compute  $(\lambda^*, \Lambda^*)$  using Eq. (4) and  $\mathcal{D}_{\text{new}}$            $\triangleright$  Dual conditioning
6:    $\mathbf{X}_b \leftarrow \mathbf{X}_b \cup \mathbf{x}_i$                                  $\triangleright \mathbf{x}_i$  is added to the current batch points
7: end for
```

Return: $\mathbf{X}_b$ $\triangleright \mathbf{X}_b$ is the chosen batch of points.

fact natural gradient updates, which [16] and [19] showed to be computationally faster than existing natural gradient methods and to have better convergence than the $(\mathbf{m}, \mathbf{V})$ parameterization. In the conditioning problem, the starting point is a set of trained SVGP variational parameters $(\mathbf{m}_{\text{old}}, \mathbf{V}_{\text{old}})$ found using $\mathcal{D}_{\text{old}} = (\mathbf{X}_{\text{old}}, \mathbf{y}_{\text{new}})$ of size n_{old} . As new data $\mathcal{D}_{\text{new}} = (\mathbf{X}_{\text{new}}, \mathbf{y}_{\text{new}})$ of size n_{new} is available, the variational parameters are updated to reflect $\mathcal{D}_{\text{old}} \cup \mathcal{D}_{\text{new}}$. Furthermore, in the conditioning problem, $\theta, \mathbf{Z}$ are assumed to be constant. In a naïve implementation, specifically using an $(\mathbf{m}, \mathbf{V})$ parameterization, it is not obvious how to solve the conditioning problem.

In dual parameterization, the problem of conditioning on new data becomes easier. We use Eq. (3) on just the new data to find $(\lambda_{\text{new}}^*, \Lambda_{\text{new}}^*)$. In conjugate models with Gaussian likelihoods, we can simply add this to the old $(\lambda_{\text{old}}^*, \Lambda_{\text{old}}^*)$, saving the need to recompute the dual parameters at old data. Thus Eq. (3) is a one-step update (i.e., $\rho = 1, t = 1$). We call this method *Dual Conditioning*. In sparse Gaussian process regression (SGPR), the change in computation of the posterior process in Eq. (1) is from $O((n_{\text{new}} + n_{\text{old}})m^2)$ to $O(n_{\text{new}}m^2)$, which can be a substantial saving when needing to fantasize and condition points in batch acquisition functions. The SGPR formulation of Dual Conditioning was first presented in [14] using the idea of pseudo-data $(\tilde{\mathbf{y}}, \tilde{\Sigma})$ which is the source parameter form of $(\lambda_{\text{old}}^*, \Lambda_{\text{old}}^*)$. Maddox et al. [13] show the usefulness of this computational saving for batch BO.

In the non-conjugate setting, [13] resorts to a Laplace approximation to find the dual parameters, which must be run for multiple iterations with a complexity² of $O((m + n_{\text{new}})^3)$. Our non-Gaussian Dual Conditioning method has a superior computational complexity of $O(n_{\text{new}}m^2 + m^3)$ per iteration and uses natural gradient variational inference, which outperforms the Laplace approximation [20]. In this setting, our previous posterior $q(\mathbf{u}; \mathcal{D}_{\text{old}})$ already contains the bulk of data, thus allowing the one-step updating of $(\lambda_{\text{new}}^*, \Lambda_{\text{new}}^*)$ even in the non-Gaussian setting ($t = 1$ and choosing $\rho = 1$ in Eq. (3)). This update step is given by Dual Conditioning updates:

$$\begin{aligned} \lambda^* &\leftarrow \lambda_{\text{old}}^* + \nabla_{\mu^{(1)}} \mathbb{E}_{q_{\mathbf{u}}(\mathbf{f}_{\text{new}})} [\log p(\mathbf{y}_{\text{new}} | \mathbf{f}_{\text{new}})], \\ \Lambda^* &\leftarrow \Lambda_{\text{old}}^* + \nabla_{\mu^{(2)}} \mathbb{E}_{q_{\mathbf{u}}(\mathbf{f}_{\text{new}})} [\log p(\mathbf{y}_{\text{new}} | \mathbf{f}_{\text{new}})]. \end{aligned} \quad (4)$$

Fig. 1 shows that using this form of Dual Conditioning outlined in Eq. (4) we are able to recover a very similar posterior to the offline solution. To build a batch for BO and AL applications, we follow the Kriging Believer algorithm (see Alg. 1 on p. 17 in [6]): we fantasize data points given from an acquisition function $\alpha(\cdot)$ and then use Eq. (4) to add the fantasized points. We outline our algorithm in Alg. 1 and optimize $\alpha(\cdot)$ using [21]. We keep inducing inputs $\mathbf{Z}$ and hyperparameters θ fixed within this inner loop of optimizing the acquisition function, but update them in the outer loop when we obtain new experimental data.

3 Experiments

To compare the fast conditioning of the dual SVGP model with that of the OVC model [13], we investigate a streaming binary classification task. We also demonstrate the benefits of our approach on a high-dimensional stochastic Bayesian optimization problem.

²See the ‘Application to BO’ section in Maddox et al. [13].

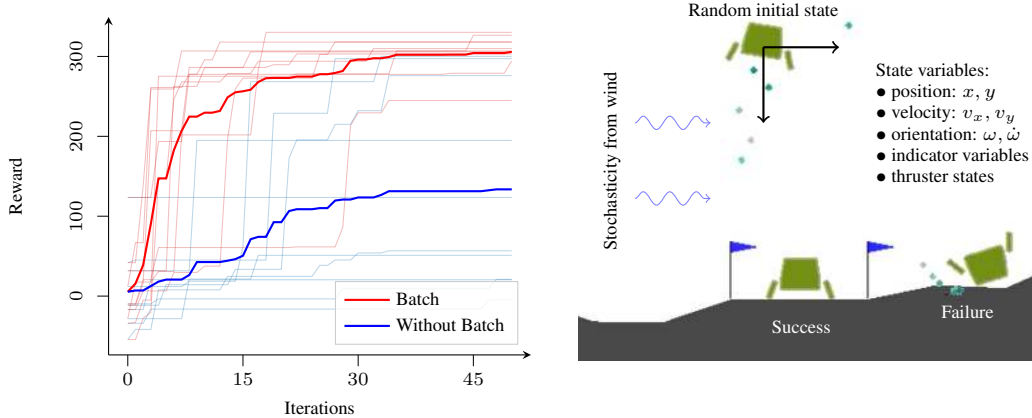


Figure 2: Bayesian optimization on a lunar landing setup; the aim is to successfully land on the surface between the flags. The stochasticity in the initial state and in the environment caused due to wind makes the problem challenging. We compare the batch solution with fantasization against the non-batch solution showcasing the benefit of batch acquisition in such noisy setups.

The streaming banana classification experiment is the same as in [13]. The data set is divided into four batches of 100 points each. In each step, only the current batch and previously inferred variational parameters are accessible; therefore, one must use an online model to condition on new data. We compare the decision boundary and predictive class probability for three models in Fig. 1. As an oracle baseline, an offline SVGP model with 25 inducing points and Matérn- $5/2$ kernel is trained on the full data, see Fig. 1a. Our method uses Dual Conditioning (Eq. (4)) on the streaming data (batches 1–4). For the OVC method, we run the code published by Maddox et al. [13]. Their method essentially initializes new models on each batch and then combines them, hence the increasing number of inducing points. The evolution of the class probability of dual SVGP and OVC is shown in Fig. 1b and Fig. 1c. The class probability obtained by the dual SVGP model after seeing the final batch closely matches that of the offline SVGP model. In contrast, the OVC method does not recover the full-data decision boundary, and its uncertainty does not match the offline baseline well.

The lunar lander problem is a challenging rocket optimization problem that aims to land successfully in a specified target region, as considered previously by [22, 23]. Here, every action performed by the lander results in a reward, and the aim is to optimize the total reward. Various environmental components add stochasticity, making it a challenging problem. The setup with the sources of stochasticity and multiple states of the lander is shown in Fig. 2. The search space spanning over $\mathbb{R}^{12}$ is high-dimensional; however, batching and fantasizing the data points can help overcome the difficulty. We model the problem using a regression model that aims to maximize the reward and a classification model for whether the landing was successful, using Alg. 1. The acquisition function is a product of Expected Improvement of the regression model and the predictive mean of the classification model. The non-Gaussian likelihood of the classification model prevents the use of many batch acquisition functions which exploit properties unique to Gaussian likelihoods, such as q-KG [8]. Our method is agnostic to the likelihood, and so allows batching and fantasizing in this challenging setup. The same set of 24 initial data points is used and optimized for 50 iterations for both models. The batch model builds batches of 20 query points. We run the experiment with 10 random initial observations and plot mean and individual rewards along with the BO iterations in Fig. 2.

4 Discussion and Conclusion

In this paper, we have unified previous work on efficiently conditioning sparse GPs on new data; our framework allows for the same variational method irrespective of the likelihood. In big-data regimes our method can be used to efficiently incorporate new information into the posterior. We show the usefulness of our technique on batch BO/AL with an experiment on a complex stochastic problem. In future work, we wish to apply our method in model-based reinforcement learning as well as extending it to updating $\mathbf{Z}$ and θ in the fully online setting discussed by [14].

References

- [1] Jonas Mockus, Vytautas Tiesis, and Antanas Zilinskas. The application of Bayesian methods for seeking the extremum. *Towards global optimization*, 2(117-129):2, 1978.
- [2] Julien Bect, David Ginsbourger, Ling Li, Victor Picheny, and Emmanuel Vazquez. Sequential design of computer experiments for the estimation of a probability of failure. *Statistics and Computing*, 22(3):773–793, 2012.
- [3] Clément Chevalier, David Ginsbourger, and Xavier Emery. Corrected kriging update formulae for batch-sequential data assimilation. In *Mathematics of Planet Earth*, pages 119–122. Springer, 2014.
- [4] David Ginsbourger, Rodolphe Le Riche, and Laurent Carraro. Kriging is well-suited to parallelize optimization. In *Computational intelligence in expensive optimization problems*, pages 131–162. Springer, 2010.
- [5] Thomas Desautels, Andreas Krause, and Joel W Burdick. Parallelizing exploration-exploitation tradeoffs in Gaussian process bandit optimization. *Journal of Machine Learning Research*, 15:3873–3923, 2014.
- [6] Clément Chevalier, Julien Bect, David Ginsbourger, Emmanuel Vazquez, Victor Picheny, and Yann Richet. Fast parallel kriging-based stepwise uncertainty reduction with application to the identification of an excursion set. *Technometrics*, 56(4):455–465, 2014.
- [7] Victor Picheny. Multiobjective optimization using Gaussian process emulators via stepwise uncertainty reduction. *Statistics and Computing*, 25(6):1265–1280, 2015.
- [8] Jian Wu and Peter Frazier. The parallel knowledge gradient method for batch Bayesian optimization. *Advances in Neural Information Processing systems*, 29, 2016.
- [9] Michalis Titsias. Variational learning of inducing variables in sparse Gaussian processes. In *Artificial intelligence and statistics*, pages 567–574. PMLR, 2009.
- [10] Sattar Vakili, Henry Moss, Artem Artemev, Vincent Dutordoir, and Victor Picheny. Scalable thompson sampling using sparse Gaussian process models. *Advances in Neural Information Processing Systems*, 34:5631–5643, 2021.
- [11] Henry B Moss, Sebastian W Ober, and Victor Picheny. Information-theoretic inducing point placement for high-throughput Bayesian optimisation. *arXiv preprint arXiv:2206.02437*, 2022.
- [12] Victor Picheny, Henry Moss, Leonard Torossian, and Nicolas Durrande. Bayesian quantile and expectile optimisation. In *The 38th Conference on Uncertainty in Artificial Intelligence*, 2022.
- [13] Wesley J Maddox, Samuel Stanton, and Andrew G Wilson. Conditioning sparse variational Gaussian processes for online decision-making. *Advances in Neural Information Processing Systems*, 34:6365–6379, 2021.
- [14] Thang D Bui, Cuong Nguyen, and Richard E Turner. Streaming sparse Gaussian process approximations. *Advances in Neural Information Processing Systems*, 30, 2017.
- [15] Henry B Moss, David S Leslie, Javier Gonzalez, and Paul Rayson. Gibbon: General-purpose information-based Bayesian optimisation. *Journal of Machine Learning Research*, 22(235): 1–49, 2021.
- [16] Vincent Adam, Paul Chang, Mohammad Emtiyaz E Khan, and Arno Solin. Dual parameterization of sparse variational Gaussian processes. *Advances in Neural Information Processing Systems*, 34:11474–11486, 2021.
- [17] Mohammad Khan and Wu Lin. Conjugate-computation variational inference: Converting variational inference in non-conjugate models to inferences in conjugate models. In *Artificial Intelligence and Statistics*, pages 878–887. PMLR, 2017.
- [18] James Hensman, Alexander Matthews, and Zoubin Ghahramani. Scalable variational Gaussian process classification. In *Artificial Intelligence and Statistics*, pages 351–360. PMLR, 2015.
- [19] Paul E Chang, William J Wilkinson, Mohammad Emtiyaz Khan, and Arno Solin. Fast variational learning in state-space Gaussian process models. In *2020 IEEE 30th International Workshop on Machine Learning for Signal Processing (MLSP)*, pages 1–6. IEEE, 2020.
- [20] Mohammad Emtiyaz Khan and Håvard Rue. The Bayesian learning rule. *arXiv preprint arXiv:2107.04562*, 2021.

- [21] James Wilson, Frank Hutter, and Marc Deisenroth. Maximizing acquisition functions for Bayesian optimization. *Advances in neural information processing systems*, 31, 2018.
- [22] Henry B Moss, David S Leslie, and Paul Rayson. Bosh: Bayesian optimization by sampling hierarchically. *arXiv preprint arXiv:2007.00939*, 2020.
- [23] Andrei Paleyes, Henry B Moss, Victor Picheny, Piotr Zulauski, and Felix Newman. A penalisation method for batch multi-objective Bayesian optimisation with application in heat exchanger design. *arXiv preprint arXiv:2206.13326*, 2022.